```
# library
class ThirdPartyLibrary::Crawler
  attr_internal :log_level
end

# client code
class MyCrawler < ThirdPartyLibrary::Crawler
  attr_accessor :log_level
end
```

In the previous example it could be the case that `:log_level` does not belong to the public interface of the library and it is only used for development. The client code, unaware of the potential conflict, subclasses and defines its own `:log_level`. Thanks to `attr_internal` there's no collision.

By default the internal instance variable is named with a leading underscore, `@_log_level` in the example above. That's configurable via `Module.attr_internal_naming_format` though, you can pass any `sprintf`-like format string with a leading `@` and a `%s` somewhere, which is where the name will be placed. The default is `"@_%s"`.

Rails uses internal attributes in a few spots, for examples for views:

```
module ActionView
  class Base
    attr_internal :captures
    attr_internal :request, :layout
    attr_internal :controller, :template
  end
end
```

Defined in `active_support/core_ext/module/attr_internal.rb`.

3.2.3 Module Attributes

The macros `mattr_reader`, `mattr_writer`, and `mattr_accessor` are the same as the `cattr_*`macros defined for class. In fact, the `cattr_*` macros are just aliases for the `mattr_*` macros. CheckClass Attributes.

For example, the dependencies mechanism uses them:

```
module ActiveSupport
  module Dependencies
    mattr_accessor :warnings_on_first_load
    mattr_accessor :history
    mattr_accessor :loaded
    mattr_accessor :mechanism
    mattr_accessor :load_paths
    mattr_accessor :load_once_paths
    mattr_accessor :autoloaded_constants
    mattr_accessor :explicitly_unloadable_constants
    mattr_accessor :logger
    mattr_accessor :log_activity
    mattr_accessor :constant_watch_stack
    mattr_accessor :constant_watch_stack_mutex
  end
end
```

Defined in `active_support/core_ext/module/attribute_accessors.rb`.

3.3 Parents

3.3.1 `parent`

The `parent` method on a nested named module returns the module that contains its corresponding constant:

```
module X
```